Homework 2

Machine Learning for Finance (21D012)
Barcelona School of Economics

Marco Canal Michael Duarte Gonçalves Olivia Peter

17 June 2026

1 Backtesting trading strategies on sentiment indicators

The sentiment columns in `dataSent` can be read as daily, target-specific sentiment indicators. At the document level a lexicon-based score for target G_k is $S_{n,t}(\lambda,k)=\sum_i w_i\, S_{i,n,t}(\lambda,k)$, and the daily indicator is the aggregation $S_t(\lambda,k)=\sum_{n=1}^{N_t}\beta_n\, S_{n,t}(\lambda,k)$ over the N_t timestamped documents of day t. Here λ selects the sentiment lexicon and k the target, i runs over the unigrams of document n with weights w_i, $S_{i,n,t}$ is the lexicon value of unigram i, and β_n weights each document in the daily total. We therefore treat positiveP, negativeP, certaintyP, uncertaintyP, finupP and findownP as observed daily sentiment time series for each stock, and we build the assignment indicators as transformations of them. Reading them this way matters: it makes the variables aggregated indicators with a meaning, not arbitrary columns, and that is what the transformations below rely on.

Our two stocks are Deutsche Bank (DB) and HSBC. Both are international banks, and both saw their adjusted-close series in `dataSent` fall sharply from January 2018 to May 2020. We flag this now because it sets a low buy-and-hold bar, a point we come back to when reading the results.

1.1 Indicators

Each daily file runs from 2 January 2018 to 22 May 2020 and supplies, beside the OHLC record, the percentage-tone scores and the news-volume measure RVT. From the tone scores we construct

$$\text{BULL}=\text{positiveP}+\text{certaintyP}+\text{finupP},\qquad \text{BEAR}=\text{negativeP}+\text{uncertaintyP}+\text{findownP},$$

$$\text{BBr}=\frac{100\,\text{BULL}}{\text{BULL}+\text{BEAR}},\qquad \text{PNlog}=\tfrac{1}{2}\log\!\left(\frac{\text{positiveP}+1}{\text{negativeP}+1}\right).$$

On the rare days where both BULL and BEAR are zero the ratio BBr is undefined. We fill those gaps by forward fill after the first valid observation so that no future information leaks back into the early sample. The explanatory-variable catalogue used in tuning is $\text{ev}\in\{\text{positiveP},\text{negativeP},\text{BULL},\text{BEAR},\text{BBr},\text{PNlog}\}$, with RVT held aside as the confirmation source.

1.2 The signal

Our signalling rule is a sentiment slope confirmed by a news-volume z-score. Given a slope lag L, an RVT window W and a confirmation threshold τ, the L-day change in the explanatory variable is $\text{slope}_t=\text{ev}_t-\text{ev}_{t-L}$ and the standardised news volume is

$$z_t=\frac{\text{RVT}_t-\mu_t^{\text{RVT}}}{\sigma_t^{\text{RVT}}},\qquad \mu_t^{\text{RVT}}=\text{mean}_{t-W:t}(\text{RVT}),\quad \sigma_t^{\text{RVT}}=\text{std}_{t-W:t}(\text{RVT}).$$

The decision at t is to go long when the slope is positive and $|z_t|>\tau$, to exit (flat in the long-only regime, weight -1 in the long-short regime) when the slope is negative and $|z_t|>\tau$, and to hold otherwise. We lag

the signal one day, so the trade at $t+1$ uses only information through t. The double condition is what matters: a tone change has to be confirmed by a burst of news before it moves the position.

Two stylised facts about news sentiment motivate this. The first is that news volume and volatility move together, so a tone change arriving on abnormal news flow is likelier to carry information than a quiet drift in the score. The second is asymmetry. Bad news tracks market moves more tightly than good news, which is why the rule will exit or short when the slope deteriorates under heavy news flow. Quiet wiggles in tone are mostly noise. Once the 1% cost is paid on every flip, trading them loses money. The volume filter is there so we act only when the news is genuinely moving.

1.3 Tuning

The free parameters are the slope lag $L \in \{5, 10, 20\}$, the RVT window $W \in \{20, 40\}$, the threshold $\tau \in \{0, 0.5, 1\}$, the choice of ev and the long-short flag. We select them by grid search over the whole sample, maximising the cumulative excess return $\mathrm{Me}_T - \mathrm{BH}_T$ subject to a floor of five trades. That floor matters. It throws out degenerate configurations that almost never trade and would otherwise score well just by sitting out a falling benchmark. We also ran three other sentiment families during development, Bollinger bands, MACD and a rolling-percentile breakout on ev. The slope-with-confirmation family held up best across both banks, and it is the one we submit.

We score the grid on the same period we then report on, so the headline tables are *in-sample model selection* followed by descriptive rolling diagnostics, not out-of-sample validation. The guard against overfitting holds only in part. The same family and the same timing, $L = W = 20$ and $\tau = 1$, are picked for both banks, and only the explanatory variable differs, BBr for DB and positiveP for HSBC. A stricter test freezes one bank's whole configuration and runs it unchanged on the other. That still clears buy-and-hold after costs: the DB rule earns a cumulative excess of 0.71 on HSBC, and the HSBC rule earns 1.35 on DB. To remove the hindsight altogether we also run a walk-forward test, which fixes the parameters on data before each evaluation window. It is reported in Section 1.6 and is the out-of-sample check.

1.4 Conventions

We measure performance by the excess return after costs. A completed long-only buy-sell trade from day b to day s has log return $r_{bs} = \log(P_s/P_b) - 2C$, where P_b and P_s are the buy and sell prices and C is the one-way cost. For a rule ρ the period return is $r = \sum_t I_\rho(t)\, r_t - 2nC$, where $I_\rho(t)$ is one on the days the rule is invested, r_t is the day-t return and n is the number of completed trades. Buy-and-hold is $r_{1T} = \log(P_T/P_1) - 2C$ with P_1 and P_T the first and last prices. The headline figure is the excess $\mathrm{Me} - \mathrm{BH} = r - r_{1T}$. We charge the cost once per position change at the rate $C = 0.01$, so the total cost over a window is (number of position changes)$\times 0.01$ taken off the cumulative return. A direct long-to-short flip therefore counts as a single position change, handled the same way for whole-period and rolling results. Two of the metrics deserve a word of explanation:

i. *Sharpe.* We report the Sharpe ratio as an annualised return over daily volatility scaled by $\sqrt{252}$. That scaling inflates it by a factor of 252 over the conventional reward-to-variability ratio $SR = (E(R) - r_f)/\mathrm{std}(R)$, with r_f set to zero, so the values run into the hundreds and are best read as relative rankings rather than standard Sharpe numbers.

ii. *Maximum drawdown.* We define it as the worst peak-to-trough drop of the cumulative-return path, $\min_t(\mathrm{cum}_t - \max_{s \le t} \mathrm{cum}_s)$, with the drawdown length the longest stretch the path spends below a prior peak. We compute that length directly as the longest underwater run, so the values reported below are non-zero.

1.5 In-sample results

Deutsche Bank and HSBC are tuned and evaluated the same way, so we report them together. The whole-period and rolling tables below are the in-sample selection, scored on the same period we tuned on; the

out-of-sample check is the walk-forward of Section 1.6.

Deutsche Bank. The tuned configuration is ev = BBr, $L = W = 20$ days, $\tau = 1.0$. Table 1 gives the whole-period metrics. In long-short form the strategy returns a cumulative 1.31 against a buy-and-hold of −0.70 over the 2.5-year sample, an excess of 2.00 after the cost on each of its 45 trades. Its Sharpe is 237 on this scaling, its annualised volatility 0.45 and its worst drawdown −0.38 lasting 123 trading days. The long-only version still beats buy-and-hold by 80 points of cumulative return, just by standing aside on weak-sentiment days. So a good part of the edge is staying out when the news turns bad, not the short leg alone.

configuration	CumRet	AnnRet	Sharpe	WinPct	AnnVol	MaxDD	DDLen	nTrades
DB long-short	1.306	0.420	236.8	0.541	0.447	−0.382	123	45
DB long-only	0.104	0.042	40.6	0.521	0.263	−0.262	258	44
DB buy-and-hold	−0.699	−0.395	−219.9	0.480	0.453	−1.071	586	1

Table 1: Whole-period DB metrics for the long-short and long-only rules against buy-and-hold (BBr, $L = W = 20$, $\tau = 1.0$), net of the 1% cost.

Table 2 reports the rolling-window analysis. At one-year windows the long-short configuration averages a positive excess of 0.819 per window with a window-by-window standard deviation of 0.216, and it beats buy-and-hold in every one of the 347 overlapping windows. The six-month windows tell the same story at a shorter horizon, with a mean excess of 0.383, and even the long-only configurations stay positive in mean excess at both lengths. Trade counts scale as expected, around 20 a year and 10 a half-year, so the cost base stays modest relative to the gain. We stress that the 347 one-year windows overlap heavily. They are far from independent experiments. So the perfect ExcessWin record describes the chosen in-sample rule across subperiods, not 347 independent tests. ExcessWin is not the WinPct column. WinPct counts positive days inside a window, ExcessWin counts windows that beat the benchmark, and the two can pull apart. The distribution of per-window excess is shown in Figure 1.

configuration	windows	exc. mean	exc. std	AnnRet	AnnVol	Sharpe	WinPct	ExcessWin	MaxDD	DDLen
w=254, long-short	347	+0.819	0.216	+0.542	0.392	338.7	0.536	1.000	−0.300	89.0
w=254, long-only	347	+0.312	0.119	+0.039	0.285	13.7	0.513	1.000	−0.228	124.7
w=127, long-short	474	+0.383	0.321	+0.670	0.391	380.4	0.531	0.920	−0.219	49.3
w=127, long-only	474	+0.141	0.169	+0.076	0.277	14.0	0.507	0.814	−0.160	64.9

Table 2: DB rolling-window metrics at one-year and six-month windows, 1% cost per window; the long-short rule beats buy-and-hold in every one-year window.

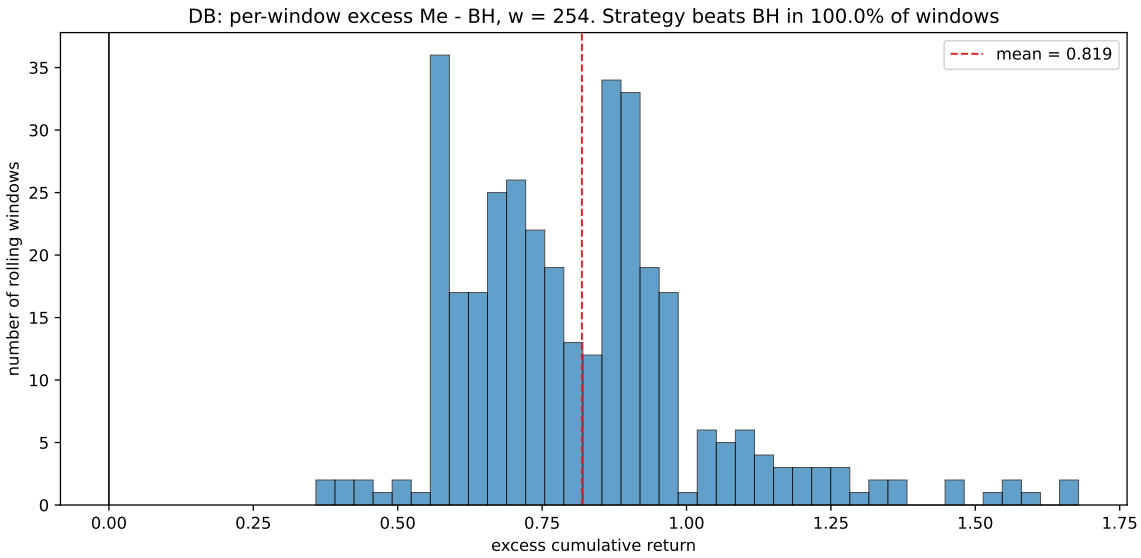


Figure 1: DB long-short per-window excess Me − BH over the one-year rolling windows; every window is positive, with a right tail from the March 2020 drawdown.

HSBC. The tuned configuration is the same family with ev = positiveP, $L = W = 20$ and $\tau = 1.0$. Only the explanatory variable changes, from the bull-bear ratio to the positive-tone score. Table 3 reports the whole-period metrics: a long-short cumulative return of 0.55 against a buy-and-hold of −0.61, an excess of 1.17 after 46 trades, with a worst drawdown of −0.11. The rolling analysis in Table 4 gives a positive mean excess of 0.294 at one-year windows with a 57% win rate, again steady across window length and the long-short flag. The six-month long-only case shows how the two win measures can diverge. It wins on most days (WinPct 0.59) yet clears buy-and-hold in under half the windows (ExcessWin 0.47).

configuration	CumRet	AnnRet	Sharpe	WinPct	AnnVol	MaxDD	DDLen	nTrades
HSBC long-short	0.552	0.202	224.9	0.563	0.227	−0.111	186	46
HSBC long-only	−0.268	−0.122	−259.9	0.553	0.119	−0.151	329	45
HSBC buy-and-hold	−0.613	−0.329	−361.0	0.490	0.229	−0.674	591	1

Table 3: Whole-period HSBC metrics (positiveP, $L = W = 20$, $\tau = 1.0$), net of the 1% cost; columns as in Table 1.

configuration	windows	exc. mean	exc. std	AnnRet	AnnVol	Sharpe	WinPct	ExcessWin	MaxDD	DDLen
w=254, long-short	347	+0.294	0.347	+0.172	0.181	222.7	0.574	0.744	−0.084	97.7
w=254, long-only	347	+0.047	0.169	−0.073	0.105	−149.1	0.591	0.582	−0.088	131.8
w=127, long-short	474	+0.170	0.234	+0.222	0.183	289.4	0.571	0.637	−0.071	51.8
w=127, long-only	474	+0.034	0.115	−0.077	0.106	−139.5	0.592	0.473	−0.064	63.6

Table 4: HSBC rolling-window metrics; all four configurations are positive in mean excess. Columns as in Table 2.

Comparison. The two equity curves (Figures 2 and 3) tell the same story, with DB the stronger case. Both strategies clear a falling benchmark after costs: DB finishes near +1.31 against an own loss of about 70% and HSBC ends near +0.55 against about 61%, excesses of 2.00 and 1.17. The DB curve sits above its benchmark almost from the start and pulls away during the March 2020 sell-off, when the short leg profits from DB falling further. The HSBC curve drifts around its benchmark through 2018 and 2019, then pulls clear as HSBC collapses in early 2020. In both the edge comes mostly from sidestepping the worst stretches and shorting into them, which is why the curves separate most sharply in the COVID quarter.

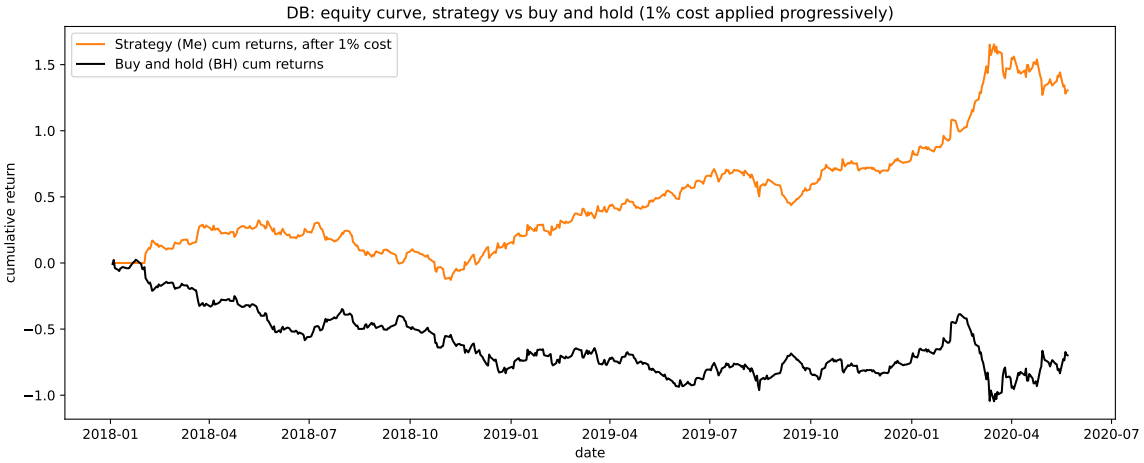


Figure 2: DB cumulative return, strategy versus buy-and-hold with the 1% cost applied progressively; the long-short rule ends near +1.31.

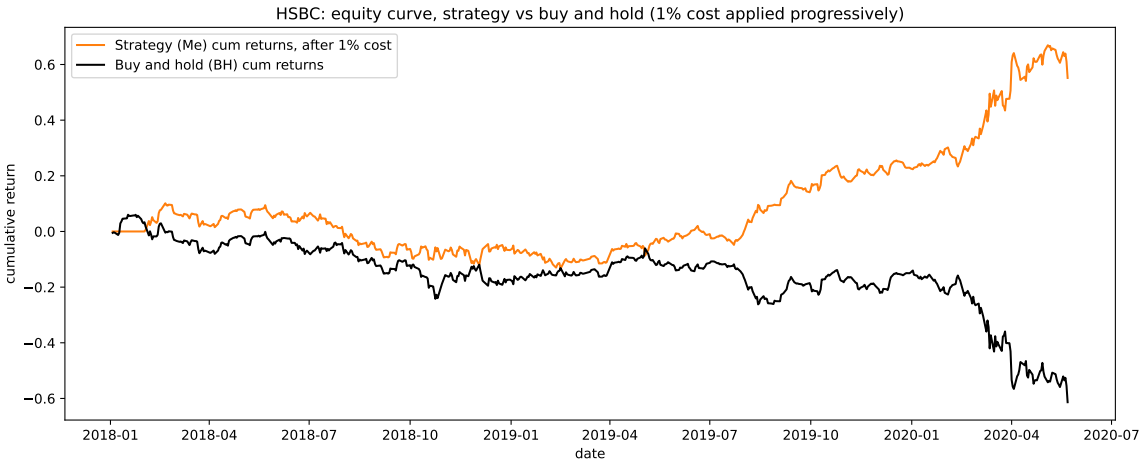


Figure 3: HSBC cumulative return, strategy versus buy-and-hold with the 1% cost applied progressively; the strategy ends near +0.55.

1.6 Walk-forward validation

The rolling tables still pick the rule with hindsight, because the grid is scored on the whole sample. A stricter and more realistic test is walk-forward, which is close to how a trading desk would actually run a signal. We tune the rule on a block of past data, here one year, freeze it, and then measure it on the next three months that the tuning never saw. We then slide the whole window forward and repeat. Each test quarter is out-of-sample, because its parameters were fixed before that quarter happened.

Table 5 shows the result for both banks across 2019 and the first quarter of 2020. The picture is weaker than the in-sample one. On DB the refitted rule beats buy-and-hold after costs in four of the five test quarters, with a mean test excess of 0.20. On HSBC it wins in only two of five, with a mean of 0.11. In both cases the average leans almost entirely on one quarter, January to April 2020, when the short leg caught the COVID crash and made 0.82 on DB and 0.72 on HSBC. Take that quarter out and the out-of-sample edge is small on DB and slightly negative on HSBC.

The train and test columns are where to look. The tuner does not always choose the submitted slope-with-confirmation rule. In the calmer quarters it often prefers a Bollinger or percentile rule on a different sentiment variable, which tells us no single configuration is reliably best once we stop looking at the answer. And the train excess, between 0.2 and 1.3, towers over the test excess of a few percent. That gap is the overfitting the in-sample tables cannot show. Taken together, the picture is plain. The signal has

a modest, regime-dependent edge, and most of the headline performance earlier is in-sample selection rather than a repeatable out-of-sample return.

ticker	test quarter	tuned rule	train exc.	test exc.	nTrades
DB	2019 Q1	Boll/PNlog	1.29	+0.03	2
DB	2019 Q2	Boll/PNlog	0.86	+0.08	2
DB	2019 Q3	Slope/BBr	0.68	−0.03	1
DB	2019 Q4	Slope/BBr	0.86	+0.09	5
DB	2020 Q1	Slope/BBr	0.59	+0.82	1
HSBC	2019 Q1	Boll/PNlog	0.35	−0.11	1
HSBC	2019 Q2	Pctile/positiveP	0.20	0.00	0
HSBC	2019 Q3	Boll/BBr	0.20	+0.10	3
HSBC	2019 Q4	Pctile/positiveP	0.40	−0.14	4
HSBC	2020 Q1	Slope/positiveP	0.27	+0.72	2

Table 5: Walk-forward out-of-sample test: each rule is tuned on the prior year and frozen, then scored on the next quarter (test exc., excess after the 1% cost); both banks lean on the March 2020 quarter.

1.7 Discussion

On DB and HSBC alike the signal clears buy-and-hold after costs. That holds at the one-year and the six-month window, and whether or not we allow shorting. Both banks lost heavily over 2018 to 2020. The buy-and-hold bar is low, so any rule that sidesteps the worst stretches starts ahead, and the short leg adds to that by catching the March 2020 crash. The confirmation filter also keeps us out when the news is quiet. That holds trading to about 20 a year and stops the 1% cost from eating the signal.

We would not read too much into it, though. This is two correlated bank stocks over two and a half years, the windows overlap, and we tuned the parameters on the same period we report. We also read the result as predictive rather than causal. The volume-confirmed sentiment slope is a signal selected by its backtest performance, not evidence that sentiment drives the next price move. The signal could easily weaken in another sector or another decade. The walk-forward test of Section 1.6 already cuts the headline down to a modest out-of-sample edge, and a fuller verdict would need other sectors or a longer history. We also charge the 1% cost inside the rolling-window evaluation, the optional extension the brief asks for, so it is already in every figure above.

2 Machine learning for option valuation

The goal is a fast stand-in for a slow pricer. We train a model on the parameter-price pairs a numerical method computes, then query it cheaply. We do this for two contracts, American Put/Call options priced on a binomial tree and down-and-out Barrier calls priced by Heston Monte Carlo, with a Gaussian process and a feed-forward network as the surrogates.

Start with the payoffs. A vanilla call pays $\max(S_T - K, 0)$ and a put pays $\max(K - S_T, 0)$ at maturity. A down-and-out call is path-dependent: it pays like a call only if the underlying never breaches a lower barrier B before maturity, and pays zero otherwise. That is why we price the American options on a binomial tree, where the only path question is when to exercise, and the barrier option by Monte Carlo, where we have to watch the whole path.

2.1 Pricing American options

We sample $n = 1000$ parameter combinations from

$$K \sim U(80,120),\ T \sim U(0.1,2),\ r \sim U(0,0.05),\ q \sim U(0,0.03),\ \sigma \sim U(0.1,0.6),$$

with the spot fixed at $S_0 = 100$, where K is the strike, T the maturity, r the risk-free rate, q the dividend yield and σ the volatility. The contract type is drawn uniformly so the dataset mixes puts and calls (the realised draw has 522 puts and 478 calls). The type enters the feature set as a binary indicator, one for a call and zero for a put. Uniform sampling is appropriate because the pricing function is smooth in these variables and even grid coverage matters more than parameter realism.

We compute each label by Cox-Ross-Rubinstein backward induction with 500 steps. With time step $\Delta t = T/500$ the up and down factors and the risk-neutral probability are $u = e^{\sigma\sqrt{\Delta t}}$, $d = 1/u$ and $p = (e^{(r-q)\Delta t} - d)/(u - d)$. At maturity the node value is the local payoff. At each earlier node the continuation value is $e^{-r\Delta t}\big(p\,C_{i+1,j} + (1-p)\,C_{i,j}\big)$, with $C_{i,j}$ the option value at tree node (i, j), and the American feature enters by replacing it with the larger of continuation value and immediate exercise value. This early-exercise comparison is exactly why a tree suits the American options. The full sample prices in about 6 seconds, 6.3 milliseconds per option.

2.2 Pricing barrier options

For the barrier call we draw $n = 1000$ Heston combinations with $K \sim U(80,120)$, $T \sim U(0.25,1)$, $r \sim U(0,0.05)$, $q \sim U(0,0.03)$, $v_0 \sim U(0.01,0.1)$, $\theta \sim U(0.01,0.15)$, $\kappa \sim U(0.5,5)$, $\eta \sim U(0.1,0.6)$, $\rho \sim U(-0.9,-0.1)$ and $B = S_0 \cdot U(0.6,0.95)$ so that $B < S_0 = 100$ holds by construction. Here v_0 is the initial variance, θ the long-run variance, κ the mean-reversion speed, η the volatility of variance, ρ the correlation and B the barrier. The labels are risk-neutral Monte Carlo prices. Monte Carlo valuation approximates a discounted expected payoff: simulate risk-neutral paths, evaluate the contract payoff on each, discount and average. Here the dynamics are

$$dS_t = (r-q)S_t\,dt + \sqrt{v_t}\,S_t\,dW_t^S, \qquad dv_t = \kappa(\theta - v_t)\,dt + \eta\sqrt{v_t}\,dW_t^v, \qquad d\langle W^S, W^v\rangle_t = \rho\,dt,$$

with v_t the instantaneous variance and W^S, W^v two Brownian motions correlated at ρ. The down-and-out payoff is zero on any path that breaches B and equals the call payoff otherwise. We integrate the price in log form and the variance with the full-truncation Euler scheme to keep the square-root terms well defined, using 100 time steps and 10,000 antithetic paths per option. The engine itself is the deliverable of Problem 3. The full sample prices in about 19 seconds, 18.9 milliseconds per option.

Two approximations are worth flagging. We monitor the barrier on the simulation grid, so strictly we price a *discretely* monitored version of the continuously monitored contract in the brief. A finer grid or a Brownian-bridge correction would shrink the bias near the barrier. And the Monte Carlo labels carry simulation noise, which we measure: across the 1000 barrier options the antithetic standard error of the price averages 0.083, with a 90th percentile of 0.137 (Table 9). That is the noise floor any surrogate is judged against.

2.3 The surrogates

Both surrogates take the raw parameter vector, standardised feature by feature on the training split, with an 80/20 train-test split fixed by seed. The Gaussian process uses a Matérn 5/2 kernel with per-dimension length scales plus a white-noise term, its hyperparameters fitted by maximum marginal likelihood. That white-noise term matters on the noisy barrier labels. With $y_i = f(x_i) + \epsilon_i$, f a Gaussian process and $\epsilon_i \sim N(0, \sigma_n^2)$, the posterior mean $\mu = K(X_*, X)\left[K(X, X) + \sigma_n^2 I\right]^{-1} y$ is the predicted price, where X stacks the training inputs, X_* is a new input, y the training labels, $K(\cdot, \cdot)$ the kernel, σ_n^2 the label-noise variance and I the identity. The noise term lets the model avoid interpolating the Monte Carlo noise on the barrier labels. The neural network is a two-layer feed-forward model with 64 tanh units per layer, trained by Adam with early stopping. We pick a feed-forward design over an LSTM (the brief allows either) because the input is a static parameter vector, on which recurrence buys nothing.

2.4 Accuracy and speed

Table 6 reports test-set accuracy and timings.

contract	model	MAE	RMSE	R^2	MAPE	fit (s)	pred (ms)	speedup	break-even
American Put/Call	GPR	0.054	0.130	0.99977	0.006	31.9	0.015	412	5,061
American Put/Call	NN	0.139	0.242	0.99920	0.019	3.3	0.0014	4,397	527
Barrier Call	GPR	0.122	0.162	0.99909	0.019	95.9	0.017	1,115	5,090
Barrier Call	NN	0.239	0.356	0.99562	0.031	2.1	0.0015	12,736	109

Table 6: Test-set accuracy (MAE, RMSE, R^2, MAPE) and timing for the GPR and NN surrogates on each contract, with speedup over the numerical pricer and the break-even query count.

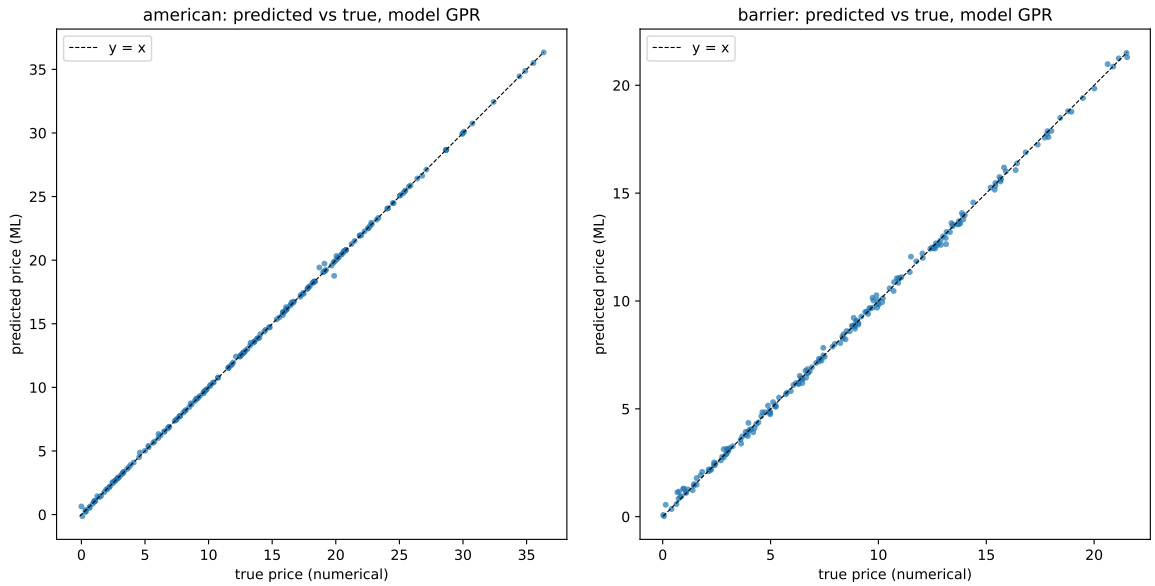


Figure 4: Predicted versus true test-set prices for the GPR model: American (left) and Barrier down-and-out Call (right); the dashed line is the identity.

It is worth looking past the aggregate numbers. Split by contract type (Table 7), the GPR error runs a little larger on puts than calls, which fits the richer early-exercise behaviour of the American put. The network is roughly even across the two. Either way the single pooled model loses little by handling both. No-arbitrage checks (Table 8) confirm the surrogates mostly respect the price bounds. Predictions are non-negative on essentially all test options, never exceed the spot ceiling for calls and dip below intrinsic value only on a handful of deep contracts where the unconstrained regressor overshoots by a few cents. None of these is enforced in training, so the small violation rates are a fair picture of where a production system

would want a clamp. One bound is barrier-specific. A down-and-out call must weakly lose value as the barrier B rises toward spot, because knock-out grows more likely. We test this by bumping B by two points on each test option and checking the predicted price does not rise. The GPR keeps the price monotone on 74% of cases and the NN on 76%. At roughly three-quarters, the surrogate is not production-safe in B, since neither model enforces the bound. A deployed pricer would add a monotonicity constraint or a post-processing step such as isotonic projection along the barrier. The check is cheap and finance-specific, and it is exactly the audit a learned pricer should pass before use.

model	subset	MAE	RMSE	R^2
GPR	put (90)	0.063	0.140	0.99972
GPR	call (110)	0.047	0.122	0.99981
NN	put (90)	0.130	0.188	0.99949
NN	call (110)	0.146	0.279	0.99899

Table 7: American GPR and NN test errors split by put and call (subset sizes in brackets).

contract	model	<0	$<$ intr.	$>$ bound
American	GPR	0.5%	2.0%	0%
American	NN	1.0%	3.0%	0%
Barrier	GPR	0%	n/a	0%
Barrier	NN	1.5%	n/a	0%

Table 8: Share of test predictions violating non-negativity, the intrinsic-value floor (American) and the upper price bound.

dataset	mean MC SE	90th-pct SE	GPR MAE
Barrier Heston labels	0.083	0.137	0.122

Table 9: Monte Carlo noise of the barrier labels: mean and 90th-percentile antithetic standard error against the GPR test MAE, which sits at the noise floor.

2.5 Discussion

On the American surface the GPR reaches an R^2 all but equal to one, with a typical error around five cents on prices that run from zero to forty, and it prices puts and calls equally well from a single model. On the barrier surface the target is not the exact Heston price but a Monte Carlo estimate, so the test residual mixes model error and label noise. The GPR MAE of 0.122 sits right at the noise floor of those labels (mean standard error 0.083, with a 90th percentile of 0.137). That means it has reached about the best a surrogate can do against noisy targets, not that it is a tenth of a euro from the true price. The network is an order of magnitude cheaper to fit and to query, at the cost of two to three times the MAE. It is the engine to reach for when latency dominates, with GPR as the accuracy benchmark when it does not.

The speedups need one clarification. The column compares one already-trained prediction against one fresh numerical pricing call, but training is a one-off cost that we only repay after enough queries. The break-even count is

$$N_{BE} = \frac{T_{\text{fit}}}{t_{\text{num}} - t_{\text{pred}}},$$

training time over the per-query time saved, *not* the number of training samples divided by the speedup. With our timings this is about 5,100 American GPR queries, 530 American NN queries, 5,100 Barrier GPR queries and just over 100 Barrier NN queries. Below those counts the numerical pricer is cheaper if no model exists yet. Above them, which is the normal regime in production where the same parameter range is queried thousands of times, the surrogate is decisively faster. The exact break-even is hardware-dependent (these timings are from the machine that produced the tables), but the ordering is robust: the cheap-to-train network repays itself fastest, the expensive GPR needs the most queries to amortise.

3 Faster Euler Monte Carlo

Every variant below estimates the same quantity, the discounted expected payoff. Under Euler discretisation a geometric-Brownian path follows $S_{t+\Delta t} = S_t + rS_t\Delta t + \sigma S_t\sqrt{\Delta t}\,\epsilon_t$ with volatility σ and $\epsilon_t \sim N(0,1)$, and the option value is approximated by $m^{-1}\sum_{i=1}^{m} f(\widehat{S}_{i,n})\,e^{-rT}$, where m is the number of paths, f the payoff and $\widehat{S}_{i,n}$ the simulated price at the final step n of path i. We split the computation into three blocks: generating the innovations, simulating the security paths and evaluating the discounted payoff. Each improvement below touches just one block and leaves the priced quantity alone. It only makes the estimator faster or cuts its variance.

3.1 The variants

i. V0 is the baseline, a doubly nested Python loop with one scalar normal per step per path.

ii. V1 keeps the same arithmetic Euler step but vectorises across paths, so the path loop disappears into a NumPy broadcast. This changes only the `SecurityPath` block.

iii. V2 adds antithetic variates, drawing $N/2$ normals Z and using Z and $-Z$, so the symmetric part of the noise integrates exactly. This changes the `OptInnovations` block.

iv. V3 replaces the arithmetic step with the log-Euler step $\log S_{t+\Delta t} = \log S_t + (r - \frac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t}\,Z$, which is exact for geometric Brownian motion and removes the discretisation bias of the arithmetic scheme.

v. V4 swaps the pseudo-random draws for a scrambled Sobol low-discrepancy sequence mapped through the inverse normal CDF, again acting on the innovations.

For Heston we also use a full-truncation Euler scheme. Writing $v_t^+ = \max(v_t, 0)$, the variance and log-price updates the code applies are

$$v_{t+\Delta t} = v_t + \kappa(\theta - v_t^+)\Delta t + \eta\sqrt{v_t^+}\sqrt{\Delta t}\,Z_t^v,$$

$$\log S_{t+\Delta t} = \log S_t + \left(r - q - \tfrac{1}{2}v_t^+\right)\Delta t + \sqrt{v_t^+}\sqrt{\Delta t}\,Z_t^S,$$

with correlated shocks $Z_t^S = \rho Z_t^v + \sqrt{1-\rho^2}\,\epsilon_t$ and $Z_t^v, \epsilon_t \sim N(0,1)$. Flooring the variance at zero inside the square roots is what keeps them well defined under discretisation. It changes the numerics, not the risk-neutral Heston model.

3.2 Vanilla benchmark

Table 10 prices a European call with $S_0 = K = 100$, $T = 1$, $r = 0.03$, $\sigma = 0.25$, whose Black-Scholes value is 11.348. The scalar baseline runs only 2,000 paths, the most that is tractable in pure Python. Vectorisation buys the dominant gain, close to two orders of magnitude per path while reaching ten times as many paths. Antithetic variates roughly halve the runtime by drawing $N/2$ shocks instead of N. For this one fixed seed the signed pricing error also falls, from +0.231 for V1 to +0.065 for V2, but a single signed error is not a variance. The true variance reduction is the one in Figure 5, where the RMS error is averaged over 20 seeds. Log-Euler shaves a little more time and removes the small arithmetic bias. The Sobol variant is slower per run because the generator costs more than a pseudo-random draw, but it converges faster in N, as Figure 5 shows.

3.3 Heston benchmark

For the Heston down-and-out call with $S_0 = K = 100$, $T = 1$, $r = 0.03$, $v_0 = \theta = 0.04$, $\kappa = 2$, $\eta = 0.3$, $\rho = -0.5$, $B = 80$ we compare the scalar naive scheme against the vectorised full-truncation engine used in Problem 2.

variant	description	price	error vs BSM	time (ms)	paths
V0	scalar loop, arithmetic Euler	11.286	−0.063	471	2,000
V1	vectorised, arithmetic Euler	11.579	+0.231	48	20,000
V2	V1 + antithetic	11.413	+0.065	24	20,000
V3	V2 + log-Euler	11.412	+0.064	20	20,000
V4	V3 + Sobol QMC	11.296	−0.052	195	20,000

Table 10: Vanilla European-call benchmark for the five variants against the Black-Scholes truth 11.348; V0 runs at fewer paths because the scalar loop is slow.

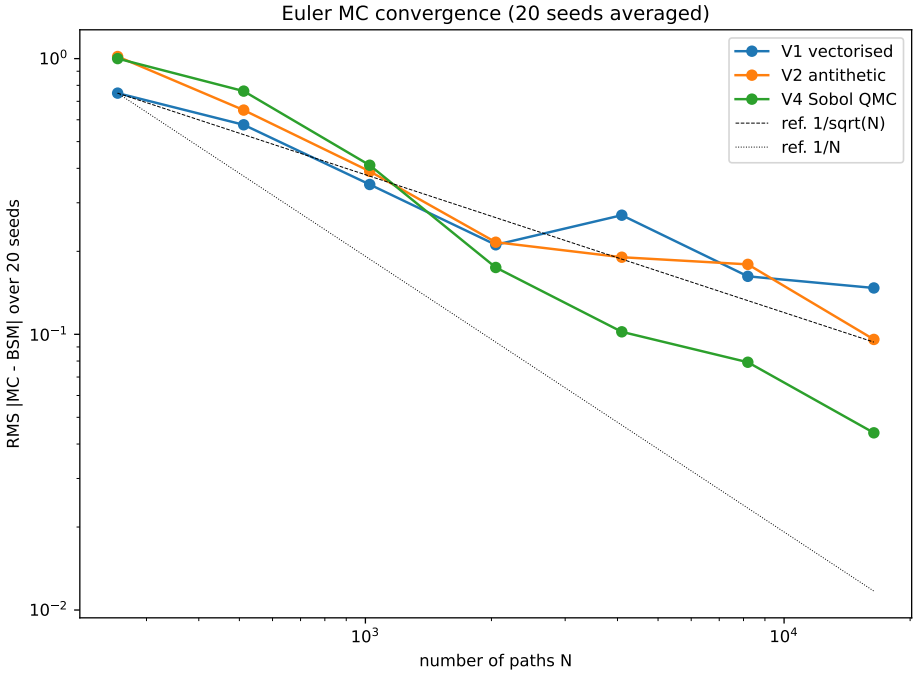


Figure 5: RMS pricing error of the vectorised variants versus path count N, averaged over 20 seeds; scrambled Sobol (V4) breaks below the $1/\sqrt{N}$ reference.

The scalar version prices 1,000 paths in 518 milliseconds, which scales to roughly 5,200 milliseconds for the 10,000 paths a stable barrier price needs. The vectorised engine prices 10,000 paths in 31 milliseconds. That is well over a hundredfold speedup, and it is what makes the 1000-option ground truth of Problem 2 feasible in well under a minute rather than tens of minutes.

3.4 Discussion

Vectorisation provides the dominant speedup, about two orders of magnitude in CPU time. Antithetic variates then add a variance reduction worth roughly half a path-count doubling. Log-Euler removes the small bias of the arithmetic scheme at no cost. And flooring the variance keeps the Heston path well behaved under discretisation. For Problem 2 we keep log-Euler with antithetic variates in the vanilla pipeline and its full-truncation analogue in the barrier engine. One caveat on Sobol. In this smooth, low-dimensional vanilla benchmark the scrambled sequence does converge faster than pseudo-Monte Carlo over the range we tested, but there is no such guarantee for the discontinuous, path-dependent barrier payoff under Heston: that would need its own check, and probably a path construction that reduces the effective dimension. So we use Sobol for the vanilla pricing and leave it out of the barrier engine.

4 Author contribution

All authors contributed equally.